```
class Lab03 {
    static void Main() {
        Console.WriteLine("Ready");
    }
}
```

Human Computer Interaction

Lab 03

Classes:

A class is a blueprint for creating objects.
It defines:

- Data that an object contains
- Behavior that an object can perform
- The structure and capabilities of a particular type

A class can contain:

- **Fields / Properties** → represent data
- **Methods** → represent behavior

Defining a class:

```
2 references
public class Cat
{
    1 reference
    public string Name { get; set; }
    1 reference
    public int Age { get; set; }

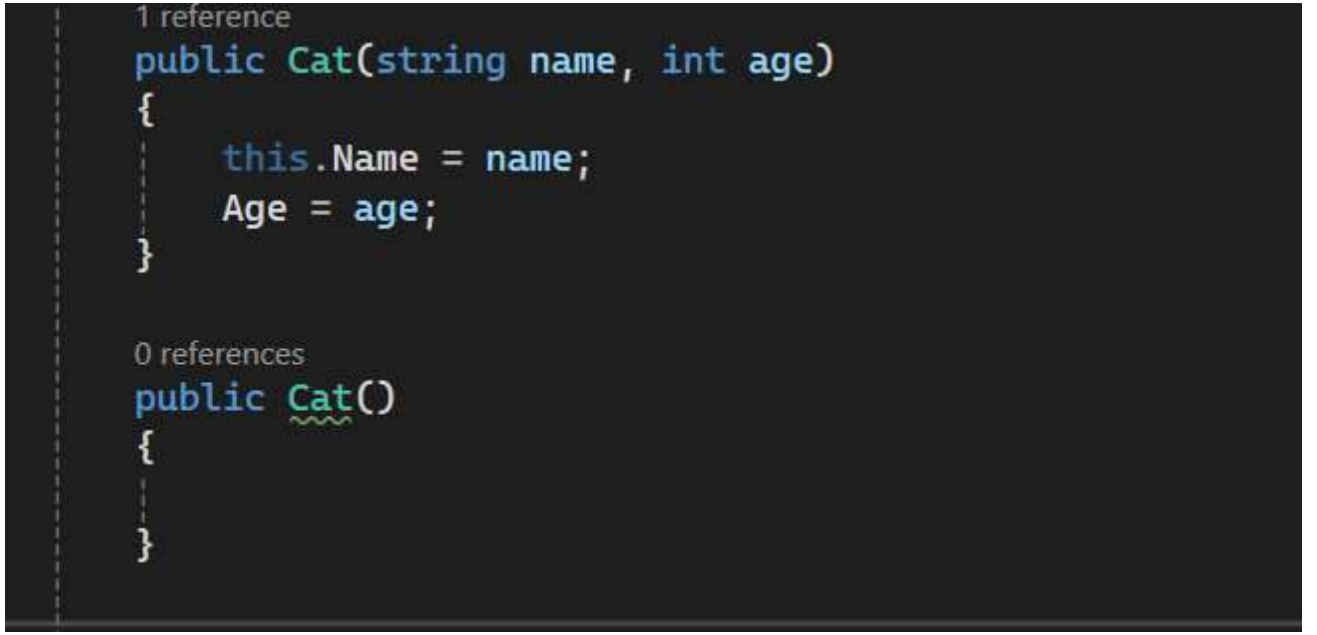

    1 reference
    public void MakeASound()
    {
        Console.WriteLine("Meow");
    }

}
```

```
Cat c = new Cat();
c.Name = "garfield";
c.Age = 1;
c.MakeASound();
```

Constructor:

A constructor is a special method used when an object is created.



Copy Constructor

A copy constructor takes an existing object of the same class and copies its fields into a new object, so the two are independent.

C# does not provide one automatically; you write it yourself.

```
class Student
{
    public string Name;
    public double Gpa;

    public Student(Student other)
    {
        Name = other.Name;
        Gpa = other.Gpa;
    }
}

Student b = new Student(a);
b.Name = "Sara";   // a.Name is unchanged
```

Constructor vs method

Constructor	Method
Same name as the class	Has its own name
No return type	Has a return type or `void`
Runs when object is created	Called explicitly
Initializes the object	Performs an operation

Creating Objects

Use the new keyword to create an object (an instance). Each object has its own copy of the fields.

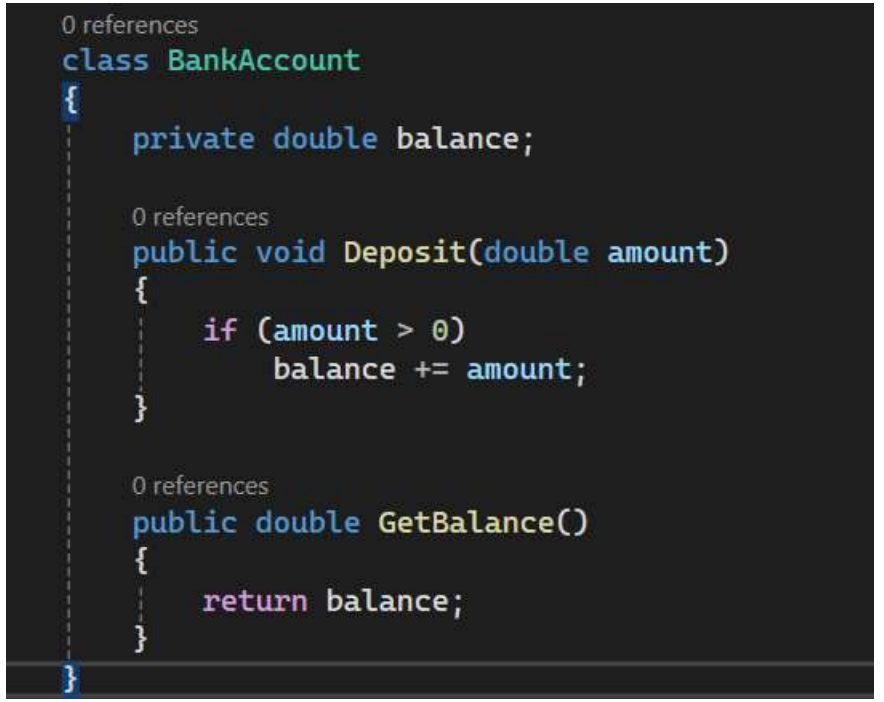
ClassName obj = new ClassName();

```
static void Main(string[] args)
{
    Student s1 = new Student();
    s1.Id = 1;
    s1.Name = "Ali";
    s1.Gpa = 3.4;
    s1.PrintInfo();

    Student s2 = new Student();
    s2.Name = "Sara";
}
```

Access modifiers:

Public:
Accessible from outside the class.
public string Name;
Private:
Accessible only inside the class.
private double balance;
Protected:
Accessible only inside the class and its derived classes;



Problem 1:

Create a Student class with:
Name
ID
Array of grades (Size 3)
Add the following methods:
CalculateAverage()
GetHighestGrade()
GetLowestGrade()
IsPassed()
PrintInfo()

Static Members and this

A static member belongs to the class itself and is shared by all objects. this refers to the current object.

Console.WriteLine(Student.Count);

```
class Student
{
    public static int Count = 0;
    public string Name;

    public Student(string name)
    {
        this.Name = name;
        Count++;
    }
}
```

What is Inheritance?

Inheritance lets you reuse code and organize classes into hierarchies

A derived class:
Reuses the fields and methods of a base class
Adds its own new members
Can change inherited behavior (overriding)
Is declared as class Student : Person
Models an is-a relationship

Terminology:
Base class = parent = superclass
Derived class = child = subclass
A Student is a Person
A Person is not necessarily a Student
C# allows only one base class
Every class inherits from object

Inheritance in C#

Put a colon and the base class name after the derived class name. The derived class gets all non-private members.

class Derived : Base { ... }

```
class Person
{
    public string Name { get; set; }

    public void Greet()
    {
        Console.WriteLine("Hi, I'm " + Name);
    }
}

class Student : Person
{
    public double Gpa { get; set; }
}
```

The base Keyword

A derived constructor calls a base constructor with base(...) so the inherited part is initialized first.

public Student(...) : base(name)

```
class Person
{
    public string Name { get; set; }
    public Person(string name) { Name = name; }
}

class Student : Person
{
    public double Gpa { get; set; }

    public Student(string name, double gpa) : base(name)
    {
        Gpa = gpa;
    }
}
```

Method Overriding

Mark the base method virtual, then replace it in the derived class with override.

Also override ToString() to control how an object prints.

```
class Person
{
    public virtual void Describe()
    {
        Console.WriteLine("I am a person.");
    }
}

class Student : Person
{
    public override void Describe()
    {
        Console.WriteLine("I am a student.");
    }
}
```

Polymorphism

A base-class variable can hold a derived object, and the overridden version runs.

Output: I am a person. / I am a student.

```
List<Person> people = new List<Person>();
people.Add(new Person());
people.Add(new Student());

foreach (Person p in people)
{
    p.Describe();
}
```

Problem 2:

Inheritance: Shapes
Create a base class Shape with a virtual method Area() that returns 0.
Then create:
Class **Rectangle** (Width, Height) with a constructor and an overridden Area().
Class Circle (Radius) with a constructor and an overridden Area().
- Optionally override **ToString()** in each class.
- Use Math.PI for the circle.

In Main, create a List<Shape> with one of each and print:
- The type of shape
- Its area
- The total area of all shapes.

Declaring a Struct

A struct looks like a class but uses the struct keyword. It is a value type, usually used for small groups of related data.

struct Name { members }

```
struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }

    public double Length()
    {
        return Math.Sqrt(X * X + Y * Y);
    }
}
```

Copy Behavior

Assigning a struct copies its data. Assigning a class copies only the reference, so both variables point to the same object.

struct = value type, class = reference type

```
Point p1 = new Point(1, 2);
Point p2 = p1;           // copy of the data
p2.X = 99;
Console.WriteLine(p1.X);  // 1

Student a = new Student("Ali");
Student b = a;           // same object
b.Name = "Sara";
Console.WriteLine(a.Name);  // Sara
```

struct vs class

Aspect	struct / class
Type	Value type / reference type
Assignment	Copies the data / copies the reference
Inheritance	None (interfaces only) / supported

When to Use a Struct

Use a struct for small, simple, ideally immutable data such as points or colors. Use a class for larger objects or inheritance.

A readonly struct cannot change after creation.

```
public readonly struct Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Problem 3:

Struct vs Class

Create a struct Point and a class PointClass, both with X and Y:

- Fields X and Y
- A constructor that sets both
- A method that prints the point

Then, in Main:

Copy each variable into a second variable and change its X.

Print the original and the copy for both types.

Explain why the results differ.